

Appointly

Appointment Scheduling Platform

Technical Specification Document

Version 1.0 | Capstone Project

Full-Stack Architecture · Schema Design · Screen Specifications · API Contracts

Table of Contents

1. Project Overview

1.1 Objective | 1.2 In Scope | 1.3 Out of Scope

2. User Roles

2.1 Customer | 2.2 Service Provider | 2.3 Admin

3. System Architecture Overview

3.1 Technology Stack | 3.2 High-Level Data Flow

4. Database Schema Design

4.1 MySQL — Relational Schema | 4.2 MongoDB — Collections

5. Appointment Lifecycle

5.1 State Definitions | 5.2 Valid Transitions | 5.3 Slot Reservation Logic

6. Authentication & JWT Specification

6.1 Token Structure | 6.2 Middleware Chain | 6.3 Auth Endpoints

7. REST API Contracts

7.1–7.7 Full API tables with request/response contracts

8. Screen-by-Screen UI Specifications

8.1 Public | 8.2 Customer | 8.3 Provider | 8.4 Admin

9. Business Logic — Availability & Booking

9.1 Slot Generation | 9.2 Conflict Detection | 9.3 Concurrency Safety

10. Notification & Email Service

10.1 Architecture | 10.2 Trigger Events | 10.3 Templates

11. Excel / Report Export Specification

11.1 Columns | 11.2 Processing Flow | 11.3 Admin Reports

12. Project Folder Structure

12.1 Frontend | 12.2 Backend | 12.3 Report Service

13. Environment Variables

13.1 Backend | 13.2 Frontend

14. Error Handling Strategy

Full error taxonomy with HTTP codes

15. Execution Plan

15.1 Week-by-Week | 15.2 Team Split

16. Evaluation Criteria Mapping

Weights per technology layer

17. Future Enhancements

Optional extensions and extra credit

18. AWS Deployment Specification

18.1 Cloud Architecture | 18.2 S3 & EC2 Config | 18.3 Simple Step-by-Step Deployment

Appendix — Key SQL Queries

Schema DDL, indexes, dashboard queries

1. Project Overview

1.1 Objective

Build a production-ready, full-stack Appointment Scheduling Platform that fulfils the following goals:

- Allow customers to discover service providers, view availability, and book appointments through a responsive web interface.
- Enable service providers to manage their profile, define availability slots, and accept or reject appointment requests.
- Provide administrators with a centralised control panel to monitor the platform, manage users, verify providers, and view operational analytics.
- Implement concurrency-safe booking with database-level constraints and transactional slot reservation to eliminate double-booking race conditions.
- Support OTP-based email verification on signup using Gmail SMTP and Razorpay payment integration for paid consultations.
- Generate downloadable Excel/PDF reports for admin operational insights.

1.2 Scope — In Scope

- User authentication — Signup with OTP email verification, Login, JWT-based session management, role-based access.
- Service provider discovery — browsable, searchable, and filterable provider catalogue with category tagging.
- Appointment booking, rescheduling, and cancellation workflows with status lifecycle tracking.
- Provider availability slot management — daily/weekly recurring templates and one-off exceptions.
- Conflict detection and concurrency-safe slot reservation.
- Customer, Provider, and Admin dashboards with role-specific KPI metrics.
- OTP email notification via Gmail SMTP on signup and appointment events.
- Razorpay payment integration for confirmed paid appointments.
- Admin panel: user management, provider verification, category management, platform analytics.
- Excel and PDF report generation via a separate Node.js report service.

1.3 Scope — Out of Scope

- Real-time chat or video conferencing between customers and providers.
- Native mobile applications (iOS/Android); responsive web only.
- Advanced AI-based provider recommendations or demand forecasting.
- Multi-tenant SaaS billing or subscription management.
- Two-way Google Calendar / Outlook sync (listed as future enhancement).

2. User Roles

The platform supports three distinct user roles, each with different access levels and capabilities:

Role	Capabilities
Customer	Sign up / Login • Browse & search providers • Book, reschedule, cancel appointments • Join slot waitlists • Purchase loyalty session packages • Fill custom intake forms • Make payments via Razorpay
Service Provider	Login • Manage profile & specialisation • Configure availability slots • Define custom services with dynamic durations/fees • Design category-specific intake forms • Accept/reject requests
Admin	Login • Manage users & suspend/verify accounts • Configure platform commission rates (5-10% split) • Manage categories, packages, and analytics • Download operational reports

2.1 Customer

- Sign up and log in using email and password. OTP verification email sent on signup via Gmail SMTP.
- Browse the provider catalogue, search by name or service, and filter by category, availability, or location.
- Book appointments by selecting a provider, choosing an available time slot, and confirming. Payment collected at this step for paid services.
- Reschedule or cancel an existing appointment subject to the provider's cancellation policy window.
- Track real-time appointment status through the lifecycle (Pending → Confirmed → Completed).
- View full appointment history with provider details, notes, and outcome.

2.2 Service Provider

- Log in using credentials issued during registration or self-signup (subject to admin approval).
- Manage their public-facing profile: name, specialisation, bio, experience, location, and profile photo.
- Configure recurring weekly availability templates (e.g. Mon–Fri 9 AM–5 PM, 30-minute slots) and one-off exceptions (holidays, closures, extended hours).
- Accept or reject incoming appointment requests with an optional rejection reason.
- View their daily and weekly calendar with all confirmed and pending appointments.
- Mark appointments as Completed or flag customer no-shows.

2.3 Admin

- Log in via a dedicated admin account created during database seeding.
- View and manage all registered users — customers and providers. Suspend, activate, or delete accounts.
- Verify new provider accounts: review credentials, approve or reject with notes.
- Manage service categories: create, rename, or deactivate categories used for provider tagging and customer filtering.

- Monitor all appointments across the platform with filtering by status, provider, date range, or category.
- Access platform analytics: total appointments, cancellation rates, provider utilisation, and peak hours.
- Download operational reports as Excel files via the Node.js report service.

3. System Architecture Overview

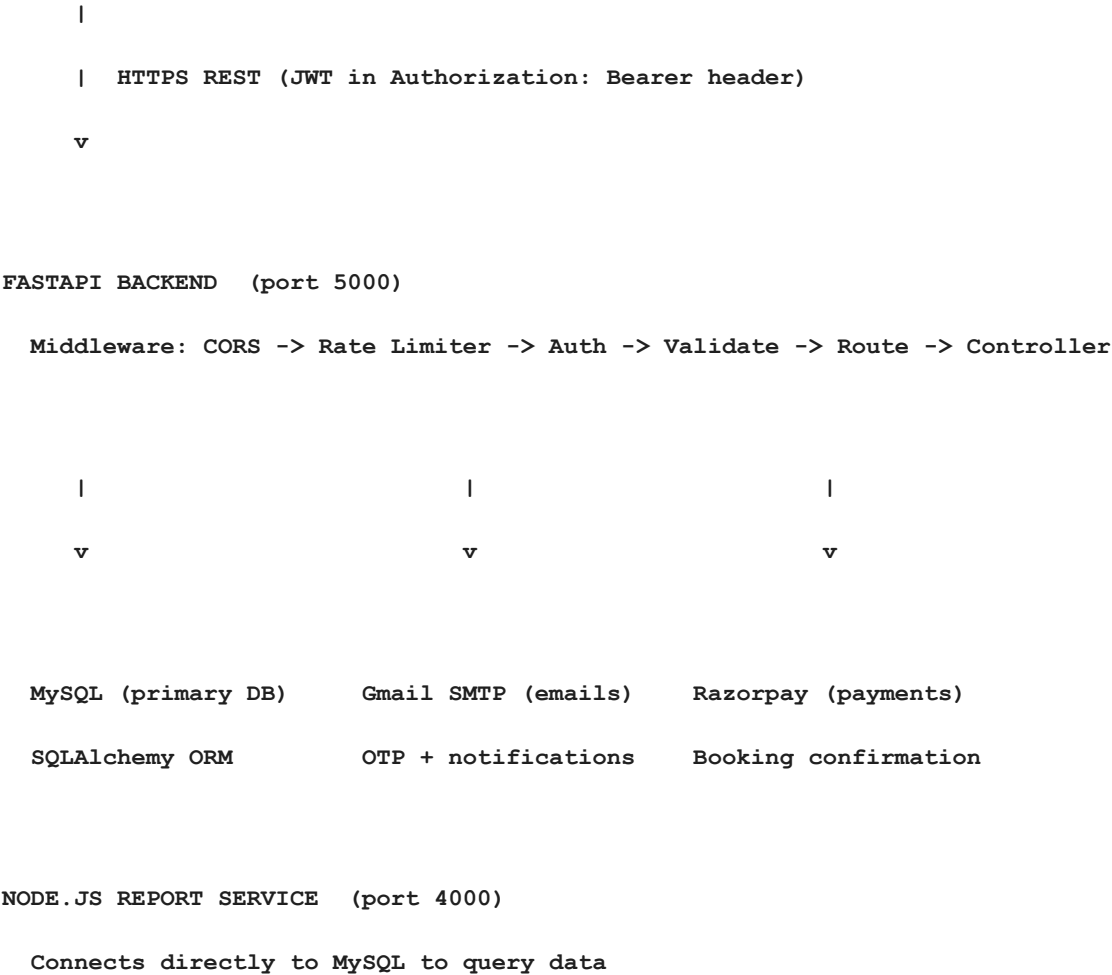
3.1 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React + Vite	18.x / 5.x	SPA with fast HMR build tooling
State Management	Redux Toolkit + RTK Query	2.x	Global state & API caching with optimistic updates
Routing	React Router DOM	6.x	Client-side SPA navigation
UI Components	Tailwind CSS + Shadcn/UI	3.x / latest	Utility-first responsive design system
Backend API	Python + FastAPI	3.11 / 0.104	Async RESTful API server with automatic OpenAPI docs
Report Service	Node.js + Express	20.x / 4.x	Dedicated Excel/PDF report generation microservice
Authentication	JWT + bcrypt + OTP	---	Stateless auth, password hashing, email OTP on signup
Primary DB	MySQL	8.x	Relational transactional data (users, appointments, slots)
ORM	SQLAlchemy	2.x	Python ORM with Alembic migrations
Data Validation	Pydantic v2	2.x	Request/response schema validation in FastAPI
Email	Gmail SMTP	---	OTP delivery on signup and appointment event notifications
Payments	Razorpay	---	Payment collection at appointment booking confirmation
Rate Limiting	Redis / In-Memory	7.x	Sliding window rate limiting; falls back to in-memory if Redis absent
HTTP Client	Axios	1.x	Frontend API calls with request/response interceptors
Report Generation	ExcelJS + PDFKit	latest	Excel and PDF report creation in the Node.js report service

3.2 High-Level Data Flow

REACT SPA (Browser) – Vite dev server / production build

Redux Store <-> RTK Query <-> Axios HTTP Client



3.3 Microservices Decoupled Architecture

The backend system uses a microservices approach to decouple heavy read/write tasks (Excel/PDF report generation) from core scheduling APIs:

- **FastAPI Application Service (Port 5000):** Acts as the primary API Gateway and transactional core. It handles user authentication, JWT verification, booking status changes, slot logic execution, and database writes to MySQL.
- **Node.js Report Service (Port 4000):** A dedicated microservice written in Express.js. It connects directly to the MySQL database and MongoDB instance. It fetches data and compiles reports dynamically via ExcelJS and PDFKit. Running reports on a separate service prevents CPU-blocking operations from degrading scheduling performance for live users.
- **Cross-Service Communication:** The React SPA sends regular scheduling and profile REST requests directly to the FastAPI backend. When an administrator triggers a report download, the frontend requests the file from the Report Service (authenticated using the same JWT tokens).

4. Database Schema Design

All tables use UUID primary keys generated by the application layer (Python uuid4). Timestamps are stored in UTC as DATETIME(6) for microsecond precision. Foreign keys are enforced at the database level with ON DELETE RESTRICT unless noted.

4.1 MySQL — Relational Schema

4.1.1 users

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4 — unique user identifier
email	VARCHAR(255)	NOT NULL, UNIQUE	Login email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash (cost factor 12)
full_name	VARCHAR(100)	NOT NULL	Display name
role	ENUM('customer','provider','admin')	NOT NULL, DEFAULT 'customer'	Access control role
phone_number	VARCHAR(20)	NULLABLE	Contact phone number
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Soft-delete / suspension flag
is_email_verified	BOOLEAN	NOT NULL, DEFAULT FALSE	Set to TRUE after OTP confirmation
otp_code	VARCHAR(6)	NULLABLE	Current active OTP (6-digit)
otp_expires_at	DATETIME(6)	NULLABLE	OTP expiry — typically now + 10 minutes
created_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Record creation timestamp
updated_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Last update timestamp

4.1.2 service_categories

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
name	VARCHAR(100)	NOT NULL, UNIQUE	Category name e.g. 'Dermatology', 'Legal Consultation'
description	TEXT	NULLABLE	Short description shown on provider listing
icon_url	VARCHAR(500)	NULLABLE	Optional icon URL for UI display

is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Soft-delete for categories
created_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Creation timestamp

4.1.3 service_providers

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
user_id	VARCHAR(36)	FK → users.id, NOT NULL, UNIQUE	Linked users record (role = 'provider')
category_id	VARCHAR(36)	FK → service_categories.id, NOT NULL	Primary service category
specialisation	VARCHAR(200)	NOT NULL	Short specialisation tag e.g. 'Orthopaedic Surgeon'
bio	TEXT	NULLABLE	Longer professional description shown on profile page
experience_years	INT	NOT NULL, DEFAULT 0	Years of professional experience
location	VARCHAR(200)	NULLABLE	City, area, or clinic address
consultation_fee	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	Fee per appointment; 0 = free
slot_duration_minutes	INT	NOT NULL, DEFAULT 30	Default appointment slot duration
profile_photo_url	VARCHAR(500)	NULLABLE	CDN URL for profile photo
is_verified	BOOLEAN	NOT NULL, DEFAULT FALSE	Set TRUE by Admin after credential review
is_accepting_appointments	BOOLEAN	NOT NULL, DEFAULT TRUE	Provider can pause bookings
rating_average	DECIMAL(3,2)	NULLABLE, CHECK(0-5)	Computed average of all customer ratings
rating_count	INT	NOT NULL, DEFAULT 0	Total number of ratings received
created_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Record creation timestamp
updated_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Last update timestamp

4.1.4 availability_slots

Each row represents one bookable time slot for a provider on a specific date. Slots are generated from the provider's weekly template (Section 9.1) and stored here for efficient query and locking.

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
provider_id	VARCHAR(36)	FK → service_providers.id, NOT NULL	Provider this slot belongs to
slot_date	DATE	NOT NULL	Calendar date of this slot
start_time	TIME	NOT NULL	Start time of the slot (UTC)
end_time	TIME	NOT NULL	End time = start_time + slot_duration_minutes
is_available	BOOLEAN	NOT NULL, DEFAULT TRUE	FALSE when booked or manually blocked
is_blocked	BOOLEAN	NOT NULL, DEFAULT FALSE	Provider manually blocked this specific slot
created_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Slot generation timestamp
UNIQUE	(provider_id, slot_date, start_time)	UNIQUE CONSTRAINT	Prevents duplicate slot generation

4.1.5 appointments

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
customer_id	VARCHAR(36)	FK → users.id, NOT NULL	Customer who made the booking
provider_id	VARCHAR(36)	FK → service_providers.id, NOT NULL	Target service provider
service_id	VARCHAR(36)	FK → provider_services.id, NOT NULL	Specific service booked (defines duration & price)
package_id	VARCHAR(36)	FK → client_packages.id, NULLABLE	Referenced loyalty package if session is prepaid
slot_id	VARCHAR(36)	FK → availability_slots.id, NOT NULL, UNIQUE	Booked slot — UNIQUE prevents double booking
appointment_date	DATE	NOT NULL	Date of appointment (denormalised from slot for query speed)
start_time	TIME	NOT NULL	Start time (denormalised from slot)
end_time	TIME	NOT NULL	End time (denormalised from slot)

status	ENUM('pending','confirmed','completed','cancelled','no_show')	NOT NULL, DEFAULT 'pending'	Current lifecycle state
notes	TEXT	NULLABLE	Customer-provided notes or reason for visit
rejection_reason	TEXT	NULLABLE	Provider rejection reason — populated on cancel/reject
payment_status	ENUM('not_required','pending','paid','refunded')	NOT NULL, DEFAULT 'not_required'	Razorpay payment state
razorpay_order_id	VARCHAR(100)	NULLABLE	Razorpay order ID for payment tracking
razorpay_payment_id	VARCHAR(100)	NULLABLE	Razorpay payment ID on successful capture
amount_paid	DECIMAL(10,2)	NULLABLE	Amount collected from customer in INR
cancelled_at	DATETIME(6)	NULLABLE	Timestamp of cancellation
completed_at	DATETIME(6)	NULLABLE	Timestamp of completion
created_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Booking creation timestamp
updated_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Last update timestamp

4.1.6 appointment_status_history

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
appointment_id	VARCHAR(36)	FK → appointments.id, NOT NULL	Parent appointment
old_status	VARCHAR(50)	NULLABLE	Previous status before transition
new_status	VARCHAR(50)	NOT NULL	Status transitioned to
changed_by_user_id	VARCHAR(36)	FK → users.id, NULLABLE	Who triggered the change
notes	TEXT	NULLABLE	Optional reason or remarks
changed_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Timestamp of transition

4.1.7 provider_availability_templates

Defines the recurring weekly availability pattern for a provider. Slots are generated from these templates for any requested date range (see Section 9.1).

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
provider_id	VARCHAR(36)	FK → service_providers.id, NOT NULL	Provider this template belongs to
day_of_week	TINYINT	NOT NULL, CHECK(0-6)	0=Monday, 6=Sunday (ISO weekday)
start_time	TIME	NOT NULL	Working start time for this day
end_time	TIME	NOT NULL	Working end time for this day
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Disable without deleting
UNIQUE	(provider_id, day_of_week)	UNIQUE CONSTRAINT	One template row per day per provider

4.1.8 provider_availability_exceptions

One-off overrides that add or block availability on specific dates — holidays, emergency closures, extended hours, or lunch breaks.

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
provider_id	VARCHAR(36)	FK → service_providers.id, NOT NULL	Affected provider
exception_date	DATE	NOT NULL	The specific calendar date
exception_type	ENUM('blocked','extended')	NOT NULL	blocked = entire day off; extended = custom hours
custom_start_time	TIME	NULLABLE	Used only when exception_type = 'extended'
custom_end_time	TIME	NULLABLE	Used only when exception_type = 'extended'
reason	VARCHAR(200)	NULLABLE	Human-readable reason e.g. 'Public Holiday'
created_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Creation timestamp

4.1.9 provider_services

Tracks the different service configurations offered by each provider, each with its own dynamic duration and pricing model.

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4

provider_id	VARCHAR(36)	FK → service_providers.id, NOT NULL	Provider who offers this service
name	VARCHAR(100)	NOT NULL	Service title e.g. 'Introductory Consult', 'Deep-dive Session'
description	TEXT	NULLABLE	Service description detail
duration_minutes	INT	NOT NULL	Slot duration of service (e.g. 15, 30, 60 mins)
price	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	Consultation fee for this service
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Toggle service availability
created_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Record creation timestamp

4.1.10 provider_custom_form_definitions

Allows each provider to define dynamic intake questions that customers must fill out during booking. The form template varies based on the category of provider.

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
provider_id	VARCHAR(36)	FK → service_providers.id, NOT NULL	Owner provider
field_name	VARCHAR(100)	NOT NULL	Unique programmatic key for input
field_label	VARCHAR(255)	NOT NULL	Display label shown to client
field_type	ENUM('text', 'paragraph', 'number', 'select', 'file')	NOT NULL	Input control type
is_required	BOOLEAN	NOT NULL, DEFAULT FALSE	Mandatory validation flag
options	TEXT	NULLABLE	JSON string array for dropdown selection items
sort_order	INT	NOT NULL, DEFAULT 0	Ordering position in checkout UI

4.1.11 appointment_waitlists

Tracks customers waiting for availability on fully booked provider calendars. Triggers auto-notifications on cancellations.

Column	Type	Constraints	Description
--------	------	-------------	-------------

id	VARCHAR(36)	PK, NOT NULL	UUID v4
customer_id	VARCHAR(36)	FK → users.id, NOT NULL	Waiting client
provider_id	VARCHAR(36)	FK → service_providers.id, NOT NULL	Target provider
slot_date	DATE	NOT NULL	Calendar date requested
preferred_start_time	TIME	NOT NULL	Preferred start window boundary
preferred_end_time	TIME	NOT NULL	Preferred end window boundary
status	ENUM('waiting', 'notified', 'expired')	NOT NULL, DEFAULT 'waiting'	Waitlist queue state
created_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Waitlist entry timestamp

4.1.12 client_packages

Stores session packages purchased by clients at a discount (e.g. 5 sessions with 15% discount), tracked on provider-client basis.

Column	Type	Constraints	Description
id	VARCHAR(36)	PK, NOT NULL	UUID v4
client_id	VARCHAR(36)	FK → users.id, NOT NULL	Client who purchased package
provider_id	VARCHAR(36)	FK → service_providers.id, NOT NULL	Provider providing the package sessions
name	VARCHAR(100)	NOT NULL	Package description name (e.g. '5 Session Bundle')
total_sessions	INT	NOT NULL	Total number of sessions included
remaining_sessions	INT	NOT NULL	Unused session balance
discount_percentage	DECIMAL(5,2)	NOT NULL, DEFAULT 15.00	Discount applied to package (e.g., 15.00 for 15%)
price_paid	DECIMAL(10,2)	NOT NULL	Total purchase price paid upfront
purchased_at	DATETIME(6)	NOT NULL, DEFAULT NOW(6)	Purchase date
expires_at	DATETIME(6)	NULLABLE	Expiration date of the package if defined

4.2 MongoDB — Collections

MongoDB is used for storing semi-structured and high-write data, specifically audit logs, system notification logs, and dynamic form submissions, providing high scalability for tracking operational history.

4.2.1 custom_form_responses collection

Stores user responses to provider intake forms. Since form fields change dynamically per provider, MongoDB is used to handle variable schemas seamlessly:

- `id` (ObjectId): Unique identifier
- `appointment_id` (String): UUID of the booked appointment
- `provider_id` (String): UUID of the service provider
- `responses` (Object/JSON): Key-value dictionary containing the form questions and answers
- `submitted_at` (ISODate): Submission timestamp

4.2.1 audit_logs collection

Tracks all actions taken by administrators and providers for compliance auditing. Each document records:

- `id` (ObjectId): Unique identifier
- `user_id` (String): UUID of the user initiating the action
- `action` (String): Specific action performed e.g., 'PROVIDER_VERIFIED', 'USER_SUSPENDED'
- `resource_type` (String): Target resource e.g., 'service_provider', 'user'
- `resource_id` (String): UUID of target resource
- `old_state` (Object/JSON): State of the resource prior to modification
- `new_state` (Object/JSON): Updated state of the resource
- `ip_address` (String): IP address of the requester
- `created_at` (ISODate): Time of action

4.2.2 notification_logs collection

Stores the delivery history of SMTP emails and OTP events:

- `id` (ObjectId): Unique identifier
- `recipient_email` (String): Target email address
- `notification_type` (String): 'OTP_SIGNUP', 'BOOKING_CONFIRMED', 'BOOKING_CANCELLED'
- `status` (String): 'sent', 'failed'
- `error_message` (String): Reason for failure if status is failed
- `payload` (Object): JSON metadata containing OTP code or booking UUID
- `sent_at` (ISODate): Timestamp of delivery attempt

5. Appointment Lifecycle

Every appointment follows a strictly defined lifecycle. All status transitions are recorded in `appointment_status_history` with a timestamp and the user who triggered the change. Terminal states (`completed`, `cancelled`, `no_show`) cannot be further transitioned.

5.1 State Definitions

#	State	Description	Triggered By
1	Pending	Customer has booked the slot. Awaiting provider acceptance. The <code>availability_slot.is_available</code> is set to <code>FALSE</code> at this point to prevent double-booking.	Customer (booking flow)
2	Confirmed	Provider has accepted the appointment request. Payment collected if <code>consultation_fee > 0</code> . Confirmation email sent to customer.	Service Provider / System (auto-confirm if provider has auto-accept enabled)
3	Completed	Appointment has taken place. Provider marks it complete. Terminal state — no further transitions.	Service Provider / Admin
4	Cancelled	Appointment cancelled before completion. The booked slot is released (<code>is_available</code> set back to <code>TRUE</code>) so it can be re-booked. Refund initiated if payment was made.	Customer (within policy window) / Provider / Admin
-	No Show	Appointment time passed with no attendance by customer. Provider marks it as no-show. Slot is not released. Terminal state.	Service Provider / Admin (automated future enhancement)

5.2 Valid Status Transitions

The table below defines permitted transitions and who can trigger them. The backend enforces these rules — attempting an invalid transition returns `HTTP 422`.

From Status	Can Transition To	Who Can Trigger
pending	confirmed, cancelled	Provider (accept/reject) or Admin
confirmed	completed, cancelled, no_show	Provider or Admin

completed	(terminal — no transitions)	N/A
cancelled	(terminal — no transitions)	N/A
no_show	(terminal — no transitions)	N/A

5.3 Slot Reservation Logic

When a customer selects a slot and submits the booking form, the following atomic transaction is executed to prevent race conditions:

```
BEGIN TRANSACTION;
```

```
-- Step 1: Lock the slot row for update (prevents concurrent bookings on same slot)
```

```
SELECT * FROM availability_slots
```

```
WHERE id = :slot_id AND is_available = TRUE
```

```
FOR UPDATE; -- Row-level exclusive lock
```

```
-- Step 2: If row not found or is_available = FALSE → ROLLBACK → return 409 Conflict
```

```
-- Step 3: Mark slot as unavailable
```

```
UPDATE availability_slots SET is_available = FALSE WHERE id = :slot_id;
```

```
-- Step 4: Create appointment record with status = 'pending'
```

```
INSERT INTO appointments (...) VALUES (...);
```

```
-- Step 5: Insert initial status history record
```

```
INSERT INTO appointment_status_history (...) VALUES ('pending', NOW());
```

```
COMMIT;
```

5.4 Waitlist Promotion & Notification Flow

When a slot is booked and fully booked, clients can register on the waitlist. Upon appointment cancellation, the system promotes and notifies waitlisted clients:

- **FIFO Selection:** The cancellation handler queries `appointment_waitlists` for the cancelled slot date/provider sorted by `created_at` ASC.
- **Status Update & Email Notification:** Sets waitlist status to 'notified', generates a Nodemailer SMTP notification to the client's email, and starts a 15-minute checkout window.
- **Automatic Expiry:** A scheduled Celery cron task checks for 'notified' waitlist entries older than 15 minutes. If no booking has occurred, status is set to 'expired', and the next waitlist client in FIFO order is promoted.

5.5 Loyalty Package Booking Deduction Flow

Clients who purchased a session bundle (e.g. 5 sessions at 15% discount) can pay using their package balance:

- **Deduction validation:** Validate `client_packages.remaining_sessions > 0` and `client_packages.expires_at > NOW()`.
- **Atomic decrement:** Run `UPDATE client_packages SET remaining_sessions = remaining_sessions - 1 WHERE id = :package_id`.
- **Zero-fee Booking:** Set `appointment.package_id = :package_id`, `appointment.amount_paid = 0.00`, and immediately mark appointment status as 'confirmed' (pre-paid).

5.6 Split Payments and Razorpay Commissions Routing

Standard online payments are settled dynamically using Razorpay Route splits:

- **Commission Calculation:** Compute platform commission (5% to 10% based on provider category or global settings).
- **Route Split Execution:** On webhook notification `/payment.captured`, Razorpay Route splits the transfer: platform commission is retained in the platform account, and the remaining 90-95% is forwarded directly to the provider's linked bank account.
- **Ledger Logging:** Record the transaction details, split commissions, and Razorpay reference IDs in the system ledger for audit.

6. Authentication & JWT Specification

6.1 JWT Token Structure

Two tokens are issued on login: a short-lived Access Token and a longer-lived Refresh Token stored in an HttpOnly cookie.

6.1.1 Access Token Payload

Header: { 'alg': 'HS256', 'typ': 'JWT' }

```
Payload: {
  'sub':      'uuid-of-user',
  'email':    'user@example.com',
  'role':     'customer' | 'provider' | 'admin',
  'full_name': 'Jane Doe',
  'iat':      1717200000,    // Issued at (Unix timestamp)
  'exp':      1717203600,    // Expires at (iat + 1 hour)
  'iss':      'appointly',
  'jti':      'unique-token-uuid' // For blacklisting on logout
}
```

6.1.2 Refresh Token

```
Payload: {
  'sub':      'uuid-of-user',
  'type':     'refresh',
  'jti':      'unique-refresh-uuid',
  'iat':      1717200000,
  'exp':      1717804800    // 7 days
}
```

Storage: HttpOnly cookie (SameSite=Strict) — NOT localStorage (prevents XSS theft)

6.2 Middleware Chain (FastAPI)

Request -> [CORS middleware] -> [Rate Limiter] -> [Body Parser]

-> [verify_jwt dependency]:

1. Extract token from 'Authorization: Bearer <token>' header
2. Decode and verify signature with JWT_SECRET
3. Check 'exp' claim — raise 401 if expired
4. Check 'jti' not in blacklist table
5. Attach decoded payload to request.state.user

-> [require_role('admin')] (admin routes only):

1. Check request.state.user['role']
2. Raise 403 if role does not match

-> Route Handler

6.3 OTP Email Verification Flow

- On signup: a 6-digit OTP is generated, stored (hashed) in users.otp_code with a 10-minute expiry in users.otp_expires_at.
- Email is sent via Gmail SMTP containing the OTP.
- Customer submits OTP to POST /api/auth/verify-otp. Backend validates OTP and expiry; on success sets is_email_verified = TRUE and clears otp_code.
- Login is blocked for unverified accounts — returns 403 with message 'Please verify your email before logging in'.

6.4 Auth API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/signup	No	Register new user; sends OTP email; returns user ID
POST	/api/auth/verify-otp	No	Submit OTP code to verify email and activate account

POST	/api/auth/login	No	Authenticate; return access token + set refresh cookie
POST	/api/auth/refresh	Refresh cookie	Issue new access token using valid refresh cookie
POST	/api/auth/logout	Yes	Blacklist jti; clear refresh cookie
GET	/api/auth/me	Yes	Return current authenticated user profile
POST	/api/auth/resend-otp	No	Resend OTP; rate-limited to 3 per hour per email
PUT	/api/auth/change-password	Yes	Update password with old password verification

6.5 Password Security

- Use bcrypt with cost factor 12 for all password hashing.
- Enforce minimum: 8 characters, 1 uppercase letter, 1 number, 1 special character.
- Never store plaintext passwords; never log passwords in any environment.
- Rate-limit login endpoint: 5 failed attempts per 15 minutes per IP address.

7. REST API Contracts

7.1 Provider APIs

Method	Endpoint	Role	Description
GET	/api/providers	Public	List providers (paginated, filterable by category, location, availability, name)
GET	/api/providers/:id	Public	Get full provider profile, rating, and available slots for the next 7 days
POST	/api/providers	Admin	Create provider record linked to a users row (role = 'provider')
PUT	/api/providers/:id/profile	Provider (self)	Update own bio, specialisation, location, photo, fee
PUT	/api/providers/:id/verify	Admin	Set is_verified = TRUE; triggers confirmation email to provider
PUT	/api/providers/:id/suspend	Admin	Set is_accepting_appointments = FALSE; reason stored
GET	/api/providers/:id/appointments	Provider (self) / Admin	List appointments for this provider (paginated, filterable by status/date)
GET	/api/providers/:id/schedule	Provider (self)	Calendar view: slots + appointments for a given week

7.2 Availability APIs

Method	Endpoint	Role	Description
GET	/api/availability/slots	Public	List available slots for a provider (filterable by date range)
POST	/api/availability/slots/generate	Provider	Generate slots based on weekly template
POST	/api/availability/exceptions	Provider	Add holiday/break exception for a specific date
DELETE	/api/availability/exceptions/:id	Provider	Delete an availability exception
PUT	/api/availability/slots/:id/block	Provider	Manually block or unblock a specific slot

7.3 Appointment APIs

Method	Endpoint	Role	Description
POST	/api/appointments	Customer	Book a slot (creates pending appointment, locks slot, returns booking details)

GET	/api/appointments/:id	Auth Users	Get appointment details, status history, and payment status
POST	/api/appointments/:id/reschedule	Customer/ Provider	Request rescheduling to a new slot
POST	/api/appointments/:id/cancel	Auth Users	Cancel appointment, release slot, initiate refund if paid
PUT	/api/appointments/:id/status	Provider/Admin	Transition appointment status (e.g. Confirm, Complete)
POST	/api/appointments/:id/payment/verify	Customer	Verify Razorpay payment signature & confirm booking

7.4 Category APIs

Method	Endpoint	Role	Description
GET	/api/categories	Public	List active service categories for provider filtering
POST	/api/categories	Admin	Create a new service category with name and description
PUT	/api/categories/:id	Admin	Update service category details
DELETE	/api/categories/:id	Admin	Deactivate service category (soft-delete)

7.5 Review / Rating APIs

Method	Endpoint	Role	Description
POST	/api/appointments/:id/review	Customer	Submit rating (1-5) and feedback review after completion
GET	/api/providers/:id/reviews	Public	Get reviews and feedback list for a provider
DELETE	/api/reviews/:id	Admin	Delete inappropriate review

7.6 Dashboard / Analytics APIs

Method	Endpoint	Role	Description
GET	/api/customer/dashboard	Customer	Retrieve customer stats, upcoming and past appointments
GET	/api/provider/dashboard	Provider	Retrieve provider KPIs, today's schedule, pending approvals

GET	/api/admin/dashboard/stats	Admin	Retrieve platform KPIs: active users, total booking volume, revenue
------------	----------------------------	-------	---

7.7 Admin APIs

Method	Endpoint	Role	Description
GET	/api/admin/users	Admin	List and search all users with role and status filters
PUT	/api/admin/users/:id/status	Admin	Activate or suspend a user account
GET	/api/admin/reports/export	Admin	Trigger download of Excel/PDF reports via Report Service

7.8 Provider Services APIs

Method	Endpoint	Role	Description
GET	/api/providers/:id/services	Public	List active services for a provider with custom durations/fees
POST	/api/provider/services	Provider	Create a new custom service offering (e.g., 15-min free call)
PUT	/api/provider/services/:id	Provider	Update service metadata, pricing, duration, or active toggle
DELETE	/api/provider/services/:id	Provider	Deactivate service (soft-delete, does not affect existing bookings)

7.9 Waitlist Management APIs

Method	Endpoint	Role	Description
POST	/api/waitlist	Customer	Join provider's waitlist queue for a specific date and time range
GET	/api/waitlist/my	Customer	List own active waitlist entries and status (waiting/notified)
DELETE	/api/waitlist/:id	Customer	Leave waitlist queue
GET	/api/provider/waitlist	Provider	View queue of waitlisted clients for scheduling planning

7.10 Custom Pre-Appointment Form APIs

Method	Endpoint	Role	Description
GET	/api/providers/:id/custom-form	Public	Fetch custom form template questions defined by the provider
POST	/api/provider/custom-form	Provider	Define or replace provider custom intake form schema fields
POST	/api/appointments/:id/form-response	Customer	Submit responses to provider custom intake form
GET	/api/appointments/:id/form-response	Auth Users	Get submitted custom form answers for an appointment

7.11 Client Package APIs

Method	Endpoint	Role	Description
GET	/api/packages	Public	List all standard package offerings available for purchase
POST	/api/packages/purchase	Customer	Purchase a session package bundle (creates client_packages record)
GET	/api/packages/my	Customer	List purchased active packages and remaining session balances
GET	/api/provider/packages	Provider	List clients holding active packages for this provider

8. Screen-by-Screen UI Specifications

8.1 Public Screens

- Landing Page: Navigation bar, hero section with search input, service categories grid, featured providers list, and call-to-action buttons.
- Search Results: Sidebar filters (category, availability date, fee range, location), sorting dropdown (rating, experience, price), and paginated provider cards showing verification badge, photo, bio, and 'Book Now' button.
- Provider Profile: Photo, rating, bio, experience years, location, consultation fee, category, and an interactive 7-day calendar interface to select available slots. Dynamically lists various service types (e.g., 15-min free intro, 60-min deep dive) which update slot availability list accordingly.

8.2 Customer Screens

- Customer Dashboard: KPI cards (Upcoming, Completed, Cancelled), list of upcoming bookings with cancel/reschedule actions, and tabular history of past bookings. Displays active session packages and remaining credits.
- Booking & Checkout: Selected slot details summary, payment gateway integration (Razorpay overlay) or 'Use Package Session' toggle, dynamic pre-appointment intake form (fields generated dynamically based on provider category/service, supporting files and text fields), client notes text area, and confirmation page.
- My Profile: Editable fields for full name, phone number, and password reset form.
- Loyalty Packages Shop: Browse discount bundles offered by providers, purchase packages using credit card/Razorpay, and view terms of expiration.
- Waitlist Portal: View status of joined waitlists, check queue position, and toggle email notification preferences for notified slots.

8.3 Provider Screens

- Provider Dashboard: Today's schedule list, pending booking approvals with Approve/Reject actions, utilization charts, and quick-toggle for booking acceptance.
- Availability Settings: Weekly schedule builder (select active days, set start/end times per day) and exception scheduler (select a date, block it or set custom hours).
- Services Manager: Define custom service offerings with custom durations (e.g. 15, 30, 60 mins) and customized prices.
- Intake Form Configurator: Drag-and-drop form field builder to define custom pre-appointment questions (text, select dropdown, paragraph, file upload) based on provider service category.
- My Profile Management: Edit bio, specialisation, location, fee, profile photo upload, and view average ratings.

8.4 Admin Screens

- Admin Dashboard: Live platform statistics (active users, bookings, revenue, cancellation rate), provider verification queue, and category manager panel.
- User Management: Searchable directory of all customers and providers with toggle actions to suspend or activate accounts.
- Report Hub: Date range selectors, category filters, and download buttons for generating Excel/PDF files.

9. Business Logic — Availability & Booking

9.1 Slot Generation Algorithm

Slots are generated in the database using the provider's template and exceptions. When the system requests slots for a provider for a date range:

- Identify active days of the week and working hours from provider_availability_templates.
- Iterate through each calendar date in the range. If the date matches an exception in provider_availability_exceptions with type 'blocked', skip slot generation for this date.
- If the exception is 'extended', use the custom start/end times instead of the weekly template.
- Generate slots by incrementing the start time by slot_duration_minutes until the end time is reached.
- Verify generated slots against existing bookings in the appointments table. Mark slot as unavailable if booked.

9.2 Conflict Detection

- Ensure slots do not overlap. The database UNIQUE constraint on (provider_id, slot_date, start_time) prevents duplicate slots.
- Enforce that booking slot start time must be in the future (greater than current system time).
- Enforce cancellation/reschedule policy: no changes permitted within 24 hours of the slot start time.

9.3 Concurrency Safety

To prevent double booking under high concurrent requests, a database-level row lock is acquired using SQLAlchemy with FOR UPDATE (or raw equivalent). The booking API handler operates in an isolated transaction block as defined in Section 5.3.

9.4 Dynamic Slot Durations & Dynamic Splitting Algorithm

Providers offer multiple services with custom durations (e.g. 15, 30, 60 minutes) in provider_services. Raw availability slots are stored in the database as fixed 15-minute intervals (e.g. 9:00, 9:15, 9:30). When listing availability for a service:

- CONTIGUOUS SCAN: The backend scans for contiguous availability_slots where is_available = TRUE for the requested date and provider.
- BLOCK FIT CALCULATION: A sliding window computes valid start times where the number of contiguous 15-minute intervals matches the service duration (e.g., 4 contiguous slots for a 60-min service).
- ATOMIC RESERVE: On checkout, all raw intervals spanning the chosen service block are set to is_available = FALSE in a single database transaction.

9.5 Automated FIFO Waitlist Promotion Algorithm

When an appointment is cancelled, the system automatically checks the waitlist to promote the next client in line:

- SLOT RELEASE: The cancelled appointment's raw slots are updated: is_available = TRUE.

- FIFO QUEUE SEARCH: Find the oldest waitlist entry: `SELECT * FROM appointment_waitlists WHERE provider_id = :p_id AND slot_date = :date AND status = 'waiting' ORDER BY created_at ASC LIMIT 1 FOR UPDATE;`
- NOTIFY & LOCK: If found, update status to 'notified', send an SMTP notification email via Nodemailer, and launch a Celery background task delayed by 15 minutes.
- TIMED EXPIRY TASK: After 15 minutes, if the waitlist entry status is still 'notified' (meaning the client did not check out the slot), change status to 'expired', and recursively run the promotion loop for the next user.

9.6 Split Payment Commission Math & Verification

Platform revenue split is processed dynamically upon payment confirmation:

- COMMISSION Math: Platform fee is calculated dynamically: $\text{commission_fee} = \text{total_amount} * \text{commission_rate}$ (where `commission_rate` is 5% to 10% based on provider category).
- PAYOUT SPLIT: $\text{payout_amount} = \text{total_amount} - \text{commission_fee}$.
- API SETTLEMENT: The platform initiates a Razorpay Route transfer request to forward the `payout_amount` to the provider's linked bank account, retaining the `commission_fee` in the platform gateway account.
- WEBHOOK LOGGING: On receipt of Razorpay webhook `payment.captured`, verify transaction signature, update `appointments.is_paid = TRUE`, and log commission ledger details in `audit_logs`.

10. Notification & Email Service

10.1 Architecture

The backend integrates with an asynchronous worker or async mail task queue to send emails without blocking HTTP requests. We configure standard SMTP parameters linking to a Gmail account with an App Password.

10.2 Trigger Events

- User Signup: Sends a 6-digit OTP code to the user's email address for account activation.
- Appointment Requested: Sends notification email to the provider informing them of a pending booking request.
- Appointment Confirmed: Sends a confirmation email to the customer with date, time, and provider details. Includes dynamic form details if custom intake form was completed.
- Appointment Cancelled: Sends notification email to the other party with the cancellation reason. Instantly initiates waitlist queue scanning.
- Waitlist Slot Notification: Sends an email to the next waitlisted client in FIFO order, providing a direct booking link valid for exactly 15 minutes.
- Package Purchase Confirmation: Sends a receipt to the client detailing the name of the package purchased, total price, number of sessions, and expiration terms.

10.3 Templates

- OTP Template: Clean layout with a bold 6-digit number, expiration warning, and security instructions.
- Booking Confirmation Template: Table details of the appointment, location/link, and instructions on how to cancel or reschedule.

10.4 Nodemailer SMTP Mail Configuration & Delivery Flow

Nodemailer is configured in the Node.js backend microservice for sending transactions emails:

- Connection Pooling: Configured with pool: true, maxConnections: 5, maxMessages: 100 to reduce connection establishment overhead.
- Secure Transport: Host: smtp.gmail.com, Port: 465 (SSL) or Port: 587 (STARTTLS). Authenticated via username and app-specific passwords.
- Failure Queueing: If Nodemailer returns a transient error, the task is retried up to 3 times with exponential backoff. Failed deliveries are logged to the MongoDB notification_logs database with error reason details.

10.5 Waitlist Promotion & Loyalty Purchase Templates

- Waitlist Promotion Template: Subject: 'A slot has opened up at Appointly!'. Content contains provider name, slot details, a countdown timer displaying the 15-minute limit, and a checkout button link.
- Loyalty Purchase Template: Subject: 'Your Appointly Session Package is Ready'. Content includes transaction ID, provider name, discount amount (e.g. 15%), total prepaid session balance, and links to book sessions.

11. Excel / Report Export Specification

11.1 Columns

The generated Excel report contains the following headers and structures:

- Appointments Sheet: Appointment ID, Customer Name, Customer Email, Provider Name, Category, Date, Start Time, End Time, Status, Consultation Fee, Payment Status, Payment ID, Created At.
- Provider Summary Sheet: Provider Name, Specialisation, Category, Total Bookings, Completed Bookings, Revenue Generated (INR), Average Rating.

11.2 Processing Flow

- Admin requests export via UI. Browser sends request to FastAPI backend with date range parameters.
- FastAPI redirects or forwards request to the Node.js Report Service.
- Report Service queries the MySQL database directly, fetches results, and compiles Excel sheet using exceljs library.
- Excel file is streamed back to the user's browser with content-type application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.

11.3 Admin Reports

- Includes overall summary metrics (KPIs) at the top of the sheets: total bookings count, total revenue, average cancellation rate, and utilization metrics.

12. Project Folder Structure

12.1 Frontend Folder Structure

```
appointly-frontend/

├─ public/

├─ src/

|   ├─ assets/           # Images, logos, icons
|   ├─ components/       # Common UI components (buttons, inputs, cards)
|   ├─ context/          # Auth and theme context providers
|   ├─ layouts/           # Shared layouts (DashboardLayout, AuthLayout)
|   ├─ pages/            # Page-level components
|   |   ├─ Customer/     # Customer dashboard, booking, profile
|   |   ├─ Provider/     # Provider dashboard, slots config, profile
|   |   ├─ Admin/        # Admin panel, user manager, reports
|   |   └─ Public/       # Home, Search, Provider profile, Login, Signup
|   ├─ store/            # Redux store configurations & slices
|   |   ├─ slices/       # Redux slices for auth, UI states
|   |   └─ apis/         # RTK Query API service definitions
|   ├─ App.jsx           # Main application component & routes config
|   ├─ index.css         # Global CSS stylesheet & Tailwind imports
|   └─ main.jsx          # App entry point
├─ package.json
└─ vite.config.js
```

12.2 Backend Folder Structure

```
appointly-backend/

├─ app/
```



```

|   |— api/                # API routes & routers
|   |   |— v1/             # API v1 routes (auth, providers, appointments, etc.)
|   |   |   |— deps.py     # Dependency injections (verify_jwt, get_db)
|   |— core/              # Core configurations, security settings, JWT helpers
|   |— db/                # Database session init & migrations config
|   |— models/            # SQLAlchemy database models mapping to MySQL
|   |— schemas/           # Pydantic schemas for validation (request & response)
|   |— services/          # Business logic services (slots gen, email sender)
|   |   |— main.py         # FastAPI application entry point
|— migrations/            # Alembic auto-generated migration scripts
|— alembic.ini
|— requirements.txt
|— README.md

```

12.3 Report Service Folder Structure

appointly-report-service/

```

|— src/
|   |— config/            # Database & environment variables settings
|   |— controllers/       # Route logic handlers for Excel and PDF gen
|   |— routes/            # Express route definitions
|   |   |— server.js      # Express app entry point
|— package.json
|— README.md

```

13. Environment Variables

13.1 Backend Environment Variables

Variable	Required Value / Example	Description
DATABASE_URL	mysql+pymysql:// root:password@localhost:3306/ appointly	MySQL DB connection string
JWT_SECRET	super-secret-key-change-in- production	Secret key used to sign JWT tokens
JWT_ALGORITHM	HS256	Encryption algorithm for signing JWTs
SMTP_SERVER	smtp.gmail.com	Outgoing email server host
SMTP_PORT	587	SMTP port (TLS)
SMTP_USERNAME	no-reply@appointly.com	Email address for notifications
SMTP_PASSWORD	app-specific-password	Google Account App Password
RAZORPAY_KEY_ID	rzp_test_abc123	Razorpay developer key ID
RAZORPAY_KEY_SECRET	abc123xyzSecretKey	Razorpay developer secret key
REDIS_URL	redis://localhost:6379/0	Redis cache URL (rate limits / cache)

13.2 Frontend Environment Variables

Variable	Required Value / Example	Description
VITE_API_URL	http://localhost:5000/api	Base URL pointing to FastAPI Backend
VITE_REPORT_SERVICE_URL	http://localhost:4000/api	Base URL pointing to Express Report Service
VITE_RAZORPAY_KEY_ID	rzp_test_abc123	Public Razorpay developer key ID

14. Error Handling Strategy

Standardized JSON responses are returned for errors. The error payload structure is: { error: { code: 'STRING_CODE', message: 'Readable message', details: [] } }.

HTTP Code	Error Code	Triggering Condition
400 Bad Request	VALIDATION_FAILED	Request body format incorrect or fails validation rules.
401 Unauthorized	INVALID_TOKEN / EXPIRED_TOKEN	Missing, expired, or invalid authorization headers.
403 Forbidden	INSUFFICIENT_PERMISSIONS	Authenticated user lacks the required role (e.g. customer access admin route).
404 Not Found	RESOURCE_NOT_FOUND	Requested user, provider, slot, or appointment does not exist.
409 Conflict	SLOT_ALREADY_BOOKED	Attempting to reserve or block a slot that is already reserved or blocked.
422 Unprocessable	INVALID_TRANSITION	Attempting to change appointment status in violation of lifecycle rules.
429 Too Many Requests	RATE_LIMIT_EXCEEDED	Exceeded the sliding window rate limit threshold.
500 Internal Error	INTERNAL_SERVER_ERROR	Unhandled server exception, database offline, or SMTP network failure.

15. Execution Plan

15.1 Week-by-Week Breakdown

- Week 1 — Initialization: Set up databases, configure FastAPI models and Alembic migrations. Develop Signup, Login, and OTP email validation. Set up Vite React template and Redux store.
- Week 2 — Provider Profile & Availability: Implement provider profile screens, weekly availability template endpoints, slot generation engine, and basic conflict checking algorithms.
- Week 3 — Booking Flow & Payments: Implement booking UI calendar components, transaction-isolated booking APIs, Razorpay SDK integration, payment capture, and status tracking.
- Week 4 — Reporting & Analytics: Develop Node.js report service, admin dashboard KPIs, and data export endpoints. Deploy staging environments and run load tests.

15.2 Team Split

- Developer 1 (Frontend): Build React views, form validations, Redux slices, Axios interceptors, calendar components, and Razorpay UI flow.
- Developer 2 (Backend): Define database schemas, SQLAlchemy models, write FastAPI route controllers, implement JWT auth middleware, email notifications, and slot locking logic.
- Developer 3 (Data/QA): Manage migrations, build the Express.js Report Service, compile test coverage, write unit tests, and coordinate environment deployments.

16. Evaluation Criteria Mapping

Rubric Element	Weight	Platform Implementation Details
Frontend Polish	25%	Responsive UI layout with Tailwind & Shadcn, robust Redux caching, global toast notifications.
Backend Architecture	25%	FastAPI async endpoints, strict Pydantic inputs validation, robust JWT security policies.
Database Design	20%	MySQL relational tables with UUIDs, indexing on search fields, concurrency-safe row-level locking.
Feature Completeness	20%	Working OTP verification, Razorpay integration, dynamic slot management, Excel reporting.
Testing & Quality	10%	Unit test coverage, error handling codes, error logs, and detailed API documentation.

17. Future Enhancements

- Two-Way Calendar Sync: Sync appointment schedules with Google Calendar and Microsoft Outlook accounts automatically.
- Virtual Consultations: Build in-app WebRTC video calling channels for remote consultation appointments.
- SMS Alerts: Integrate Twilio API for sending instant text reminders to reduce client no-show rates.
- Multi-Tenant billing: Add subscription pricing plans (Stripe) for providers to register clinic locations and staff members.

18. AWS Deployment Specification

18.1 Cloud Architecture Overview

This platform is deployed on AWS using a decoupled microservices architecture to ensure high availability, load separation, and scalable performance:

- Vite React SPA: Hosted statically on AWS S3 with CloudFront CDN distribution.
- FastAPI Backend: Deployed on a secure EC2 instance (e.g., t3.medium running Ubuntu 22.04 LTS) behind an Application Load Balancer (ALB).
- Express Report Service: Deployed on a separate EC2 instance (t3.micro) to isolate the CPU-heavy Excel/PDF generation workload from core APIs.
- Relational DB: Amazon RDS running MySQL 8.0 in a multi-AZ configuration.
- NoSQL DB: Amazon DocumentDB (MongoDB-compatible) for audits and notification logging.
- Media Assets: S3 Bucket configured with private read-write access for user-uploaded profile pictures.

18.2 S3 & EC2 Detailed Configuration

18.2.1 AWS S3 Bucket Setup

- Bucket 1: 'appointly-frontend-assets' for hosting build output files. Enabled Static Website Hosting. Configured CloudFront CDN origin point to this bucket.
- Bucket 2: 'appointly-user-uploads' for user profile photos and generated PDF documents. Set block public access policies and access permissions using S3 Pre-signed URLs for secure document access.

18.2.2 EC2 Security Groups Configuration

- FastAPI EC2 Security Group: Allow port 80/443 (HTTP/HTTPS) from ALB only; allow port 22 (SSH) from Admin Bastion Host/IP.
- Node.js Report Service Security Group: Allow port 4000 from FastAPI Backend instance only; allow port 22 for administrative SSH.
- RDS Security Group: Allow port 3306 (MySQL) from FastAPI and Report Service security groups only.
- DocumentDB Security Group: Allow port 27017 from FastAPI and Report Service security groups only.

18.3 Step-by-Step Deployment Steps

18.3.1 Backend FastAPI Service

Step 1: Provision Ubuntu 22.04 EC2 instance and connect via SSH

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install python3-pip python3-venv git nginx -y
```

Step 2: Clone repository and configure virtualenv

```
git clone https://github.com/capstone/appointly-backend.git /opt/appointly-backend
```

```
cd /opt/appointly-backend && python3 -m venv venv
```

```
source venv/bin/activate && pip install -r requirements.txt
```

```
# Step 3: Setup Gunicorn systemd service config (/etc/systemd/system/fastapi.service)

[Unit]

Description=Gunicorn FastAPI Application Server Server

After=network.target

[Service]

User=ubuntu

WorkingDirectory=/opt/appointly-backend

ExecStart=/opt/appointly-backend/venv/bin/gunicorn app.main:app -w 4 -k
uvicorn.workers.UvicornWorker -b 127.0.0.1:5000

Restart=always

[Install]

WantedBy=multi-user.target


# Step 4: Start and enable systemd service

sudo systemctl daemon-reload && sudo systemctl start fastapi && sudo systemctl enable fastapi
```

18.3.2 Node.js Express Report Service

```
# Step 1: Provision second Ubuntu EC2 instance, install Node.js v20

curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

sudo apt install -y nodejs git


# Step 2: Clone repository and install npm packages

git clone https://github.com/capstone/appointly-report-service.git /opt/appointly-report
cd /opt/appointly-report && npm install --production


# Step 3: Install PM2 daemon manager globally and start app

sudo npm install pm2 -g

pm2 start src/server.js --name appointly-report-service

pm2 startup systemd && pm2 save
```

18.3.3 Nginx Reverse Proxy Setup

Nginx is configured on the FastAPI EC2 instance to redirect external traffic securely:


```
server {  
    listen 80;  
    server_name api.appointly.my-domain.com;  
  
    location / {  
        proxy_pass http://127.0.0.1:5000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
    }  
}
```

Appendix — Key SQL Queries

A.1 Index Optimization

```
CREATE INDEX idx_users_email ON users(email);

CREATE INDEX idx_providers_category ON service_providers(category_id);

CREATE INDEX idx_slots_provider_date ON availability_slots(provider_id, slot_date,
is_available);

CREATE INDEX idx_appointments_customer ON appointments(customer_id);

CREATE INDEX idx_appointments_provider_date ON appointments(provider_id, appointment_date);

CREATE INDEX idx_waitlist_provider_date_status ON appointment_waitlists(provider_id,
slot_date, status, created_at);

CREATE INDEX idx_packages_client_provider_active ON client_packages(client_id, provider_id,
remaining_sessions, expires_at);
```

A.2 Admin Dashboard Utilization Query

```
SELECT

    sp.id AS provider_id,

    u.full_name AS provider_name,

    COUNT(a.id) AS total_appointments,

    SUM(CASE WHEN a.status = 'completed' THEN 1 ELSE 0 END) AS completed_appointments,

    SUM(a.amount_paid) AS total_revenue_inr,

    (COUNT(a.id) / NULLIF(COUNT(s.id), 0)) * 100 AS utilization_rate_percent

FROM service_providers sp

JOIN users u ON sp.user_id = u.id

LEFT JOIN availability_slots s ON sp.id = s.provider_id

LEFT JOIN appointments a ON s.id = a.slot_id

GROUP BY sp.id, u.full_name;
```

A.3 Client Package Usage & Balances Query

```
SELECT

    cp.id AS package_id,

    cp.name AS package_name,

    u.full_name AS client_name,

    sp_u.full_name AS provider_name,
```

```

    cp.total_sessions,
    cp.remaining_sessions,
    cp.discount_percentage,
    cp.price_paid,
    cp.purchased_at,
    cp.expires_at
FROM client_packages cp
JOIN users u ON cp.client_id = u.id
JOIN service_providers sp ON cp.provider_id = sp.id
JOIN users sp_u ON sp.user_id = sp_u.id
WHERE cp.client_id = :client_id
    AND cp.remaining_sessions > 0
    AND (cp.expires_at IS NULL OR cp.expires_at > NOW(6))
ORDER BY cp.purchased_at ASC;

```

A.4 FIFO Waitlist Selection Query

```

SELECT
    w.id AS waitlist_id,
    w.customer_id,
    u.full_name AS customer_name,
    u.email AS customer_email,
    w.preferred_start_time,
    w.preferred_end_time,
    w.created_at
FROM appointment_waitlists w
JOIN users u ON w.customer_id = u.id
WHERE w.provider_id = :provider_id
    AND w.slot_date = :slot_date
    AND w.status = 'waiting'
    AND w.preferred_start_time <= :slot_start_time
    AND w.preferred_end_time >= :slot_end_time
ORDER BY w.created_at ASC

```

LIMIT 1 FOR UPDATE;